Grow360: Smart Employee Performance Tracker

Database Schema Documentation

Table of Contents

System Overview

Grow360 is a comprehensive employee performance management system designed specifically for software companies. It streamlines goal setting, performance reviews, skill assessments, and provides intelligent analytics for better decision-making.

Key Objectives

- Automate performance review processes
- Enable transparent goal tracking
- Facilitate 360-degree feedback
- Provide actionable insights through analytics
- Support hierarchical organizational structures

Module Architecture

1. Authentication & Access Control Module

Purpose: Secure user authentication and role-based access control

Core Tables:

- `system_roles` - Defines access levels (HR Admin, Manager, Employee, etc.)
- `user_accounts` - User login credentials and session management

Key Features:

- Multi-level permission system
- Secure password hashing
- Session management
- Role-based UI rendering

2. Organizational Structure Module

Purpose: Company hierarchy and employee management

Core Tables:

- `departments` - Software company departments (Engineering, QA, DevOps, etc.)
- `roles` - Job positions with salary ranges and requirements
- `teams` - Development teams and squads
- `employees` - Complete employee profiles
- `employment_history` - Career progression tracking

Key Features:

- Manager-reportee relationships
- Department and team organization
- Salary range management
- Employment history tracking

3. Project Management Module

Purpose: Software project tracking and team assignments

Core Tables:

- `projects` - Software development projects
- `project_assignments` - Employee-project mappings with roles

Key Features:

- Technology stack tracking
- Resource allocation management
- Project timeline tracking
- Role-specific assignments

4. Goals & Performance Module

Purpose: Goal setting, tracking, and performance measurement

Core Tables:

- `goal_categories` - Technical, Delivery, Quality, Leadership, etc.
- `goals` - Individual employee goals with progress tracking
- `goal_milestones` - Breaking down goals into smaller tasks
- `goal_progress_logs` - Historical progress updates

Key Features:

- Quarterly and annual goal cycles
- Weighted goal importance
- Milestone-based tracking
- Progress visualization

5. Performance Review Module

Purpose: 360-degree feedback and structured performance evaluations

Core Tables:

- `review_cycles` - Review periods (Quarterly, Annual, etc.)
- `review_templates` - Role-specific review questionnaires
- `performance_reviews` - Individual review instances
- `review_responses` - Detailed feedback responses

Key Features:

- Anonymous feedback support
- Role-based review templates
- Multi-source feedback (Self, Manager, Peer, Direct Report)
- Structured rating systems

6. Skills Management Module

Purpose: Competency tracking and development planning

Core Tables:

- `skill_categories` - Technical, Soft Skills, Leadership, etc.
- `skills` - Individual skills with proficiency levels
- `employee_skills` - Employee skill assessments
- `skill_development_plans` - Personalized improvement plans

Key Features:

- Self and manager skill assessments
- Skill gap identification
- Development plan creation
- Training recommendation engine

7. Analytics & Reporting Module

Purpose: Performance insights and trend analysis

Core Tables:

- `performance_metrics` - KPI calculations and measurements
- `one_on_ones` - Manager-employee meeting tracking

Key Features:

- Performance trend analysis
- Goal achievement rates
- Team performance comparisons
- Meeting effectiveness tracking

8. System & Audit Module

Purpose: System monitoring and compliance

Core Tables:

- `activity_logs` - Complete audit trail
- `notifications` - System alerts and reminders

Key Features:

- Complete action logging
- Notification management

- Compliance reporting
- Security monitoring

Database Schema

Core Entity Relationships

```
Departments (1) ←→ (M) Roles ←→ (M) Employees
    ↓                    ↓
Teams (1) ←→ (M) Employees ←→ (M) Goals
    ↓                    ↓
Projects (1) ←→ (M) ProjectAssignments  GoalMilestones
    ↓                    ↓
Employees ←→ PerformanceReviews ←→ ReviewResponses
    ↓
EmployeeSkills ←→ SkillDevelopmentPlans
```

Key Constraints and Validations

Employee Hierarchy:

- Self-referencing foreign key in `employees.manager_id`
- Prevents circular reporting relationships
- Ensures organizational hierarchy integrity

Goal Management:

- Weightage validation (0-100%)
- Progress percentage constraints
- Date validation (start_date < due_date)

Performance Reviews:

- Rating scale validation (1-5)
- Review cycle date constraints
- Anonymous feedback handling

Skills Assessment:

- Proficiency level enumeration
- Assessment date tracking

- Development plan timeline validation

Access Control Matrix

Role Permissions

System Role	Employee Data	Goal Management	Reviews	Reports	Admin Functions
Super Admin	All employees	All goals	All reviews	All reports	Full system access
HR Admin	All employees	View/edit all	Manage cycles	All reports	User management
Manager	Direct reports	Team goals	Team reviews	Team reports	Team settings
Employee	Own profile	Own goals	Self-reviews	Own reports	Profile updates
Project Manager	Project team	Project goals	Project reviews	Project reports	Project settings

Data Access Rules

Hierarchical Access:

- Managers can view/edit data for all direct and indirect reports
- HR can access all employee data across departments
- Employees can only access their own performance data

Review Visibility:

- Self-reviews: Visible to employee and manager
- Manager reviews: Visible to employee, manager, and HR
- Peer reviews: Anonymous options available
- 360-degree reviews: Aggregated visibility

Sample Data Structure

Organizational Hierarchy

```
TechCorp Ltd.
├── Engineering Department (Head: John Doe - Director)
│   ├── Frontend Platform Team (Lead: Sarah Wilson)
│   │   ├── Priya Sharma (Senior SWE)
│   │   ├── Raj Patel (SWE)
│   │   └── David Kumar (Junior SWE)
│   └── Backend Services Team (Lead: Mike Johnson)
│       ├── Alex Chen (Lead SWE)
│       ├── Emily Davis (SWE)
│       └── Lisa Wong (Junior SWE)
├── Quality Assurance (Head: James Taylor - QA Manager)
│   ├── Anna Garcia (Senior QA Engineer)
│   └── Tom Brown (QA Engineer)
├── DevOps & Infrastructure (Head: Maria Rodriguez - DevOps Lead)
│   └── Chris Lee (Senior DevOps Engineer)
└── Human Resources (Head: Jennifer Smith - Director)
    └── Robert Jones (Senior HR Business Partner)
```

Sample Goal Categories for Software Company

Technical Goals:

- Code quality improvement
- Technology adoption
- Architecture contributions
- Bug reduction targets

Delivery Goals:

- Sprint velocity
- Story point completion
- Release cycle adherence
- Feature delivery timelines

Quality Goals:

- Test coverage improvement
- Code review participation
- Documentation quality
- Security compliance

Leadership Goals:

- Mentoring activities
- Knowledge sharing
- Process improvements
- Team collaboration

Implementation Guidelines

Database Setup

1. **Create Database**:

```sql
CREATE DATABASE grow360_production;
CREATE USER grow360_user WITH ENCRYPTED PASSWORD 'secure_password';
GRANT ALL PRIVILEGES ON DATABASE grow360_production TO grow360_user;
```

2. **Enable Extensions**:

```sql
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

3. **Execute Schema**:
    - Run the complete schema script
    - Insert master data (departments, roles, sample employees)
    - Create indexes for performance optimization

Security Considerations

Data Protection:

- Encrypt sensitive fields (salary, personal information)
- Implement row-level security for multi-tenant scenarios
- Use parameterized queries to prevent SQL injection

Access Control:

- Implement JWT-based authentication
- Use role-based middleware for API endpoints

- Log all data access and modifications

Backup Strategy:

- Daily automated backups
- Point-in-time recovery capability
- Cross-region backup replication

Performance Optimization

Indexing Strategy:

- Primary keys on all UUID fields
- Composite indexes on frequently queried columns
- Partial indexes for filtered queries

Query Optimization:

- Use materialized views for complex reporting queries
- Implement connection pooling
- Cache frequently accessed reference data

API Design Recommendations

RESTful Endpoints:

```
GET /api/employees/{id}/goals - Get employee goals
POST /api/goals - Create new goal
PUT /api/goals/{id}/progress - Update goal progress
GET /api/reviews/cycles/{id} - Get review cycle data
POST /api/reviews - Submit performance review
GET /api/reports/team/{teamId} - Get team performance report
```

GraphQL Schema (Alternative):

- Single endpoint with flexible queries
- Nested data fetching for complex relationships
- Real-time subscriptions for notifications

Monitoring and Analytics

Key Metrics to Track:

- Goal completion rates by department/team
- Review completion timelines
- Employee engagement scores
- Skill development progress
- Performance trend analysis

Dashboard KPIs:

- Overall company performance score
- Department-wise goal achievement
- Review cycle completion rates
- Skill gap analysis
- Employee satisfaction trends

Conclusion

This database schema provides a robust foundation for building a comprehensive employee performance management system. The modular design ensures scalability, security, and maintainability while supporting the complex requirements of modern software companies.

The schema is designed to:

- Support hierarchical organizational structures
- Enable comprehensive performance tracking
- Facilitate data-driven decision making
- Ensure data security and compliance
- Provide flexibility for future enhancements

For implementation support or customization requirements, refer to the detailed technical specifications and API documentation.